

International Journal of Parallel, Emergent and Distributed Systems

ISSN: 1744-5760 (Print) 1744-5779 (Online) Journal homepage: www.tandfonline.com/journals/gpaa20

Adaptive operator selection in heuristic optimization utilizing generalized experience with reinforcement learning

Mehmet Emin Aydin, Rafet Durgut, Abdur Rakib & Khoa Phung

To cite this article: Mehmet Emin Aydin, Rafet Durgut, Abdur Rakib & Khoa Phung (20 Dec 2025): Adaptive operator selection in heuristic optimization utilizing generalized experience with reinforcement learning, International Journal of Parallel, Emergent and Distributed Systems, DOI: 10.1080/17445760.2025.2605297

To link to this article: https://doi.org/10.1080/17445760.2025.2605297

© 2025 The Author(s). Published by Informa UK Limited, trading as Taylor & Francis Group.

Published online: 20 Dec 2025.

Submit your article to this journal

Article views: 724

View related articles

View Crossmark data

Citing articles: 1 View citing articles

Full Terms & Conditions of access and use can be found at
https://www.tandfonline.com/action/journalInformation?journalCode=gpaa20

INTERNATIONAL JOURNAL OF PARALLEL, EMERGENT AND DISTRIBUTED SYSTEMS
https://doi.org/10.1080/17445760.2025.2605297

Taylor & Francis
Taylor & Francis Group

OPEN ACCESS Check for updates

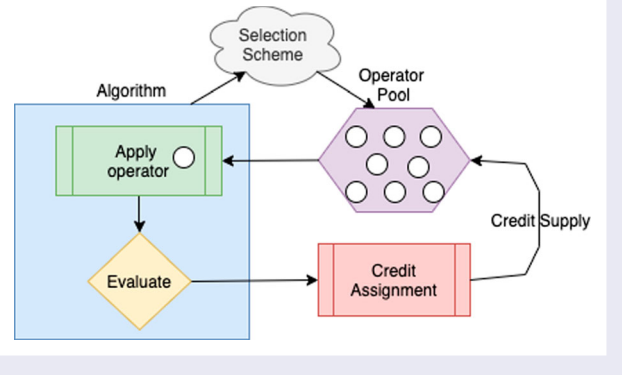
Adaptive operator selection in heuristic optimization utilizing generalized experience with reinforcement learning

Mehmet Emin Aydin[a,b], Rafet Durgut[c], Abdur Rakib[d] and Khoa Phung[a]

[a]UWE Bristol, School of Computing and Creative Technologies, Frenchay Campus, Coldharbour Lane, Bristol, UK; [b]Dept. of Industrial Engineering, Istanbul Commerce University, Istanbul, Türkiye; [c]Crystalloids BV, Rotterdam, Netherlands; [d]Centre for Future Transport and Cities (CFTC), Coventry University, Coventry, UK

ABSTRACT

Combinatorial optimization problems are inherently complex and difficult to solve. Although evolutionary and swarm intelligence algorithms have proven effectiveness, achieving an optimal balance between exploration and exploitation (EvE) remains a significant challenge. The growing trend of employing multiple operators with adaptive operator selection schemes aims to address EvE issues; however, there remains a significant demand for utilizing tailored adaptive selection schemes in search. Reinforcement Learning (RL) has recently been proposed as an efficient technique to customize and develop highly effective and adaptive selection mechanisms. However, scalability continuous to pose a significant challenge. This paper presents a novel RL-based approach that establishes a generalized and scalable framework for acquiring, processing, and leveraging experiential knowledge for both immediate and long-term use. The experimental results provide evidence of a certain level of success in validating the proposed framework.



ARTICLE HISTORY
Received 17 June 2025
Accepted 10 December 2025

KEYWORDS
Operator selection; reinforcement learning; transfer learning; general problem solver; generalized experience

1. Introduction

One of the key challenges in optimization problems is maintaining a balance between exploration and exploitation (EvE) during the search process. A potential solution to this

CONTACT Mehmet Emin Aydin mehmet.aydin@uwe.ac.uk UWE Bristol, School of Computing and Creative Technologies, Frenchay Campus, Coldharbour Lane, Bristol, UK

© 2025 The Author(s). Published by Informa UK Limited, trading as Taylor & Francis Group.
This is an Open Access article distributed under the terms of the Creative Commons Attribution-NonCommercial-NoDerivatives License (http://creativecommons.org/licenses/by-nc-nd/4.0/), which permits non-commercial re-use, distribution, and reproduction in any medium, provided the original work is properly cited, and is not altered, transformed, or built upon in any way. The terms on which this article has been published allow the posting of the Accepted Manuscript in a repository by the author(s) or with their consent.

challenge is the multiple operating neighbourhood structure, which enforces the use of various operators based on their contextual suitability. The search process involves a series of decision-making phases, during which an operator is selected to perform a move operation on the current solution. Both operator ordering and online selection necessitate adherence to specific rules. Popular approaches in the literature include probability matching [1], adaptive pursuit [2], and variations of multi-armed bandit methods [3]. These techniques have been effectively implemented in numerous studies [4, 5]. However, these methods often require executing a fixed rule applicable to all scenarios, which typically does not overlook the dynamically changing nature of the search environment and for unforeseen changes or dynamic situations. This clearly enforces to investigate if there is a generalizable approach that accounts dynamically changing search envisionment. Meanwhile, machine learning and data-driven approaches seen in the literature look promising and well-suited for addressing these dynamically changing circumstances.

It is known that the applications of search and optimization come up with a number of constraints, which need to be taken into consideration. Implementation of optimization algorithm, the methodology of problem representation, size and dimension of the domain make each application very unique and hardens replication to different problems without reconfiguration and re-setting up. A generalizable operator selection approach addressing all these issues offer a unique procedure to overcome the these challenges and help search to be more efficient. A metaheuristic algorithm including swarm intelligence and evolutionary algorithms can be made generic for solving as many problems as possible without resetting it if a generic representation can be achieved and the size and dimension constraints can be relaxed, which will make the approach generalized and scalable.

Reinforcement learning has recently been considered to integrate into a swarm intelligence algorithm so as to support the developed optimization solution with gaining experiences. It is used to work online alongside the search process and train the operator selection agent so that it can learn when and how to select an operator depending on the problem state [6]. The problem to optimize was represented in binary and the operators are binary operators, too. Although binary representation and operators are generalizable, the problem state and representation remain constrained with the problem size and dimension. This imposes restrictions on working with problem instances in the same size, not larger or smaller. The emerging issue is that how to transfer the experiences gained over the trainings in the past with certain sized problems to new cases with different sizes and dimensions. That appears as a scalability issue with the approach presented in [6]. Meanwhile the authors of [7] and [8] have implemented RL for very the same purposes but did not address the scalability and size constraint. Likewise, [9–11] present relevant work, but not touching scalability and size issues.

The problem state can be extended and enriched with more search-space related aspects to enhance the applicability of the approach. It is noted that, the problem state does not include any information related to the fitness landscape and the circumstances in the neighbourhood, which might be helpful in mapping states to actions with reinforcement learning. The works presented in [12] and [13] discuss the suitability and diversity of fitness landscape information in search efficiency enhancement. The problems state extension may help facilitate the targeted scalability. In addition, this may help map the problems state to action more effectively as it uses the search space information in a wider context. For instance, the probability of moving to a neighbouring state in better quality is much

easier in the earlier stages of search space, while it turns to be much harder in the very last stage. The search space can be more conveniently represented so that such nuances can be utilized to distinguish between the problem states. Once the process has been successfully managed in these regards, the gained experience would be ready to generalize in order for the agents to transfer their knowledge and expertise to handle and solve other similar problems.

The novelty in the work proposed in this paper is to integrate reinforcement learning into operator selection engine with a generalized approach so as to enable it learn how and when to select an operator subject to search circumstances. Then, it would be able to gain experience continually, to facilitate a generalized and scalable learning approach and be able to offer experience-based adaptive operator selection scheme. The significance is embedded in its potential to improve adaptive operator selection, providing a flexible and scalable framework for being able to reuse generalized experience for various optimization problems.

The following contributions have been achieved in this paper: (i) multi-operator neighbourhood functions facilitated with reinforcement learning to achieve generalization; (ii) feature-based input space to characterize and represent the problem states to resolve the scalability issue; (iii) search space is re-defined subject to the stages of the optimization process; and (iv) learning/trained agents are enabled to transfer their experiences across a number of problem instances without size restriction.

The rest of the paper is organized as follows. Section 2 introduces the basic underlying concepts alongside an overview of the related works, Section 3 describes the proposed approach, Section 4 provides the details of the experimental results along with relevant discussions and analysis, and finally Section 5 concludes the paper by critically evaluating the results and outlining directions for the further work.

2. Background and literature review

The proposed study combines a number of approaches to design an adaptive and generalizable search strategy that helps improve search efficiency in problem solving. A swarm intelligence algorithm, namely artificial bee colony algorithm, is embedded with a multi-operator neighbourhood function furnished with reinforcement learning to help exploit gained experiences in search. In order to keep the article self-contained, this section provides an overview of the necessary background information before moving on to a review relevant to the proposed work. Operator selection, which is the selection of the most suitable operator for a specific search problem, is crucial in optimization problems because the problem (instance) being solved has a significant impact on the algorithm's performance and the correctness of the result. However, when utilizing machine learning approach for operator selection, the performance of the operator selection model depends on the algorithm designed to use and the data to train and test the model. Since operator selection and transfer learning are the subject of this study, this is where our review and discussion will be focused.

2.1. Artificial bee colony

Artificial bee colony (ABC) is one of rather recently developed swarm intelligence algorithms that is used in this study as the problem solving framework [14]. It is a rather recent

and widely used algorithm that turned to be success proven in solving many real [15] and artificial problems [16]. The framework is created inspiring of natural collectivism (collective behaviour) of honey bees in which a population of solutions is adopted to imitate a bee colony, where different types of bees are retreated in different ways so as to keep the population diverse and efficiently approximating. The structure of the algorithm is advantageous in that the mechanic of the algorithm does not impose any compulsory rules. Due to these algorithmic properties, ABC facilitates use of multiple operators with very high capacity. Since this study only uses ABC framework as a search algorithm and does not pose the central role of this research, we preferred to present the details of a typical ABC algorithm in the appendix. Pseudo code for the ABC algorithm implemented for this study and its complexity statement are provided in the appendix with Algorithm 1.

2.2. Reinforcement learning

Reinforcement Learning (RL) is the process of learning which action to perform in order to maximize a numerical reward signal [17]. In the recent past, RL has become a widely used machine learning technique to formulate numerous engineering and scientific problems. The four fundamental components of the RL process are an action, a state, a reward, and an environment. In order to receive a reward, an RL agent acts at an appropriate time in an environment. The RL agent, however, is initially unaware of whether the action will have a positive or negative reward. Hence, the agent runs an exploration process to examine all possible states and behaviours. The agent receives reward as a result of each action performed in the preceding step. The agent evaluates the reward and stores it for later use. The agent can also exploit its knowledge and choose an action with the highest expected reward. The learning process is continued until it finds an optimal policy. Designing reinforcement learning systems presents a significant challenge in balancing the trade-off between exploration and exploitation.

2.3. Operator selection problem

The architecture of the underlying combined system and the information flow can be seen in Figure 1. As stated before, the goal of this study is to establish a generalized approach that enables search/optimization algorithms to employ immediate, temporary, and generalized experience while using population-based heuristic optimization techniques, particularly to handle combinatorial problems. The optimization algorithm considered in this study – which will be referred to as *Algorithm* henceforth – is a standard Artificial Bee Colony (ABC) algorithm [14], which is one of recently developed successfully tested swarm intelligence algorithms using population of solutions. The proposed *Algorithm* has two important components: *Apply operator* and *Evaluate* as seen in Figure 1, which apply a selected operator to the solutions under consideration and then evaluate with respect to the fitness with the objectives.

Let $\mathcal{O}$ be a finite set of operators and $\mathcal{X}$ be the solution space. The Algorithm $\mathbf{A}$ is defined as $\mathbf{A} = \{A_t \mid t = 0, 1, \ldots, T\}$, a process of T number of iterations – each represented with A_t and uses a tuple of $\langle o_i, \mathbf{x}_j, F\rangle$. Here, $o_i \in \mathcal{O}$ is an operator, $\mathbf{x}_j \in \mathcal{X}$ is a solution, and $F : \mathcal{X} \to \mathbb{R}$ is the fitness function, defined as $F(\mathbf{x}_j) = \|\mathbf{x}_j - o_i(\mathbf{x}_j)\|$, where $o_i(\mathbf{x}_j) = \mathbf{x}_{j+1}$. $F(\mathbf{x}_j)$ measures the quality of the solution of $\mathbf{x}_j$, which is also known as a fitness function. The other two

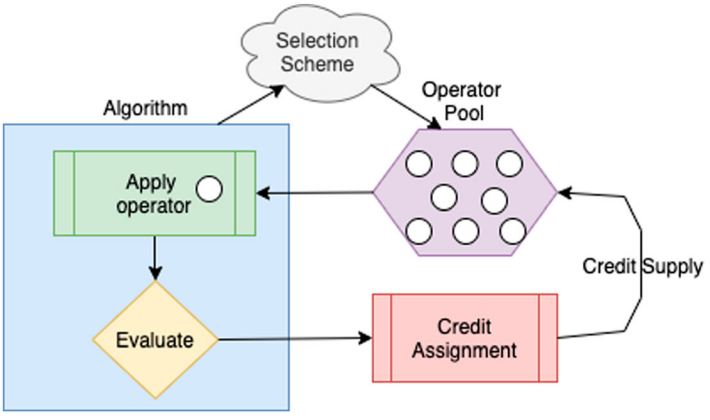


Figure 1. The overall information flow through the algorithm and other components including operator selection process.

key components of **A** are *Credit Assignment* and *Operator Pool* with which each operator $o_i \in \mathcal{O}$ is selected, used and evaluated, a credit $c_i \in \mathcal{C}$ is assigned to them for the usage in the future. Where $\mathcal{C} = \{c_i | i = 1, 2, \ldots, |\mathcal{O}|\}$, $c_i = \sum_{t=0}^{\pi} c_{i,t}$, π is the current iteration, and $c_{i,t}$ is awarded based on $F(\mathbf{x}_j)$. A *Selection Scheme* is a function $\sigma : \mathcal{X} \times \mathbb{R} \rightarrow \mathcal{O}$ defined by $\sigma(\mathbf{x}_j, F(\mathbf{x}_j)) = \arg\max_{o_i \in \mathcal{O}}\{\mathcal{C}\}$ and used to prioritize the operators and select the most suitable one. The production of $c_{i,t}$ and the implementation of $\sigma(.)$ appear to be the main areas of focus. That is, these two points are mostly the focus of the research in this area. In order to address the intended problem, the approach proposed in this article uses reinforcement learning techniques.

2.4. Related work

This section reviews the state-of-the-art research on adaptive operator selection and reinforcement learning. Adaptive operator selection (AOS) is an emerging paradigm that tries to automatically choose the best reproduction operator in accordance with the most recent search dynamics. Since the early 1990s, researchers have investigated adaptive operator selection, focusing on how it applies to evolutionary algorithms [18]. The past decade has seen a considerable advancement in the development of operator selection schemes, which are employed to enhance the efficiency of evolutionary algorithms used to address multi-objective optimization problems [19, 20]. The authors of [21] have brought forward the possibility of using reinforcement learning in operator selection first time, but not much details released.

The differential evolution with landscape-based adaptive operator selection algorithm, which chooses optimal differential evolution mutation strategies from a pool of five operators using both a landscape measure and a performance history metric, was introduced in [22]. The authors looked into the consequences of the suggested algorithm's parameters, including population size, minimum population size, and cycle length. They have evaluated the proposed algorithm's performance by solving 45 unrestricted optimization problems. However, without using any structured learning method, the proposed approach selects operators based on their performance and fitness landscape.

In order to better identify the best appropriate crossover operator, the work in [23] introduced an adaptive operator selection mechanism that combines a set of four fitness landscape analysis methodologies with an online learning algorithm called dynamic weighted majority. That is their approach suggests training a dynamic regression model for online learning to develop an operator selection scheme using fitness landscape analysis metrics.

In [24], the authors present a differential evolution method with Q-Learning (DE-QL) for engineering design problems (EDPs). The proposed DE-QL generates the trial population using the QL approach, where QL guides the choice of the mutation strategy among four different strategies as well as crossover and mutation rates from the Q table. If the (finite) number of states is kept low enough to tabulate each state against each action, their approach provides good results. However, working with Q-Table-based learning techniques is not feasible in comparatively larger state spaces.

In a rather recent work [6], the authors propose an adaptive operator selection strategy based on reinforcement learning, where the operators are mapped to the problem states according to the operation success rate. The Hard-C-Means clustering method and Q-Learning have been combined to create a reinforcement learning algorithm that helps the search agent learn how to choose the appropriate operator online based on the given situation. The authors have evaluated the proposed method in comparison to existing adaptive selection methods for addressing set union knapsack problems that are formalized in a binary representation. The results show that, in compared to well-known adaptive techniques, the proposed approach provides significantly better performance. However, the approach seems to face scalability problem due to binary representation dependent on problem size.

In [25], the authors propose another operator selection method based on reinforcement learning for evolutionary multi-objective optimization problem. Their proposed method suggests potential operators to evolve the population for various multi-objective optimization problems by utilizing deep neural networks to learn the relationships between decision variables and Q values of candidate operators during the optimization process. The action-value function, which is always defined based on the discount sum of future rewards, may be approximated using the conventional Q-learning method because the action space is discrete. In this case, the reward is based on how much the performance of the offspring improved as a result of the use of a genetic operator. The expected cumulative reward of an action given a state is indicated by each grid in a Q-Table that stores the reward associated to these actions. However, the proposed approach simply considers the decision variables of a single solution as a state. In a different approach, according to Bernoulli Thompson sampling, a Beta distribution is established for each operator in [26] and updated, after which the operators are chosen based on the rewards sampled from the distributions. In this approach, the dynamic Thompson sampling is used as the foundation for adaptive operator selection.

More recently, Johnn et al. [27] have explored the application of graph-based reinforcement learning for selecting operators from a pool to address routing problems using adaptive large neighbourhood algorithms. Similarly, Karimi-Mamaghan et al. [28] propose an iterated greedy search algorithm that incorporates Q-learning to select perturbation operators for flow shop scheduling problems, while Yi and Qu [29] present an algorithm design approach based on reinforcement learning. Although there has been a notable

growth in the application of reinforcement learning for adaptive operator selection, to the best of our knowledge, no one has yet addressed the scalability challenges in problem representation, which is expected to ease generalization of experience that is to facilitate the potential use of transfer learning to develop a more generalized approach.

Although a number of studies including [7–11] have been conducted around various forms of adaptive operator selection, none of them tackle scalability problem and size constraints. This paper attempts mainly to address emerging scalability problems in this regard.

3. Reinforcement learning in operator selection

The impact that *operator selection problem* has on the effectiveness and success of problem-solving algorithms has drawn researchers' attention for a long time. A bespoke operator selection scheme (OSS) plays the key role in balancing the exploration and exploitation activities throughout the search process. Although a number of operator selection approaches [3, 30] have been thoroughly investigated, there are still many challenges and open problems to be addressed. Recently, bespoke OSSs have been developed for higher efficiency using RL techniques [6, 31].

Reinforcement learning is a paradigm of online learning paradigm that implements active learning in a more structured way, especially when data is either not available or is only temporarily available. Obviously, it is challenging to gather search data and rely on it to create systems for data-driven decision-making. Operator selection is a procedure that is integrated into search operations and makes a selection decision based on inputs from the search environment. This enforces the adoption of reinforcement learning in order to empower operator selection using the data-driven mechanics.

One of the well-known RL algorithms is Q learning, which takes $s \in \mathcal{S}$ as the current state of a problem and estimates $Q(s, a)$, the level of the quality measure, once action $a \in \mathcal{A}$ is applied to the problem state, where $Q : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$. Once the action a is performed in the state s, a new state s' is generated, and the expected quality of the new state s' is estimated using $E(Q(s', a))$ as a stochastic function subject to the selected policy, which is often implemented as:

$$E(Q(s', a)) = \arg\max_{a \in \mathcal{A}} Q(s', a) \tag{1}$$

Following each action a taken over state s, a Bellman equation-based implementation value-iteration is used to update the corresponding quality level, and it is represented as:

$$Q(s, a) = Q(s, a) + \beta\Big(r + \gamma\, Q(s', a) - Q(s, a)\Big) \tag{2}$$

where r is the reward generated to evaluate/reward the action a, β is the learning rate, and γ is known as the discount factor. It is important to note that r is the credit – positive or negative – estimated using a reward function, $R(s, a)$, and the objective of an RL algorithm is to choose actions that maximize the agent's expected cumulative reward.

The estimation of $Q(s, a)$ is handled via a data structure called the Q-Table, where Q values are tabulated in cells of a table of $\mathcal{S} \times \mathcal{A}$. Whenever the cell of (s, a) is activated, the relevant Q value is updated using Equation (2) [32]. However, this approach is not scalable or sufficiently dynamic. There have been other proposed approximation methods, such

as feed-forward neural networks [33] and clustering algorithms [34]. The Hard-c-Means algorithm was used by the authors of [34] to set up cluster centres in conjunction with reinforcements, and they have developed a quality estimation method based on $\|s - c_a\|$, the distance between state s and c_a as the cluster centre for action a, where both $s \in \mathcal{S}$ and $c_a \in \mathcal{C}$ represent multi-dimensional data points of the same size (i.e. $dim(s) = dim(c_a)$), $|\mathcal{C}| = |\mathcal{A}|$ and c_a is the cluster centre allocated to the action a.

The clustering algorithm works as follows. First of all, $|\mathcal{A}|$ number of clusters are established, and every cluster centre is initialized to 0. It runs alongside Q learning and the agent can choose the action with the best fit a, apply it to the current state s, and generate s' accordingly. In this setting, if the action a results in higher quality, state s will be categorized as c_a, the cluster for action a. The centre for cluster c_a will be updated accordingly. The $Q(s, a)$ value will be updated using Equation (2). Given that both $s = \{s_i | i = 1 \dots dim(s)\}$ and $c_a = \{c_{a,i} | i = 1 \dots dim(c_a)\}$ are two vectors of the same size, the centre is updated as:

$$c_{a,i} = \frac{c_{a,i} + s_i}{n_a} \quad \forall i \in \{1, \dots, dim(s)\} \tag{3}$$

where n_a is a counter that counts how many times action a has been chosen correctly. As the algorithm iterates, the $Q(s, a)$ values are retrieved with $Q(s, a) = \|s - c_a\|$ as the distance between the state s and the cluster centre c_a, where the more data flows in the cluster centres are further fine-tuned.

The operator selection problem, where the problem is to choose the best operator for the given situation, has been introduced in Section 2.3. Given the set of operators $\mathcal{O}$ to be selected for the current state of the problems, $\mathbf{x}_t$ subject to the credit level used by the selection scheme $\sigma(.)$. In order to implement the RL algorithm explained above, the set of operators is adopted as the action list $\mathcal{A} = \mathcal{O}$, and the selection criteria is adopted to be $Q(\mathbf{x}_j, o_i)$ retrieved from the corresponding cluster centre $Q(\mathbf{x}_j, o_i) = \|\mathbf{x}_j - c_{o_i}\|$. Therefore,

$$\sigma(\mathbf{x}_j, F(\mathbf{x}_j)) = \begin{cases} \arg\max_{o_i \in \mathcal{O}} Q(\mathbf{x}_j, o_i) & \text{if } rnd \geq \epsilon \\ \text{random}(o_i \in \mathcal{O}) & \text{otherwise} \end{cases} \tag{4}$$

where $rnd \in \mathbb{R}$ is a random value in $[0, 1]$, random(.) is random-selection function and ϵ is a threshold – usually $\epsilon \leq 0.2$ – setup for an ϵ-greedy policy to keep exploration and exploitation harmonized through the learning process.

One of the key elements of reinforcement learning algorithms is reward generation, which determines how the environment will react to the actions the learning agent takes. In this case, the operator selection scheme is the learning agent since it learns how to choose the best operator under the given circumstances. The main aim is to maximize the accumulated reward, $\mathcal{R} = \sum_{t=1}^{\infty} r_t$, where r_t is the reward generated at iteration t for one of the operators. In fact, r_t is defined to be $r_t = R(s, a)$, which needs to be implemented for the operator selection problems. Each operator, $o_i \in \mathcal{O}$, is required to accumulate and maximize its reward, $r_{o_i,t}$, which transforms r_t into a vector of rewards, $r_t = \{r_{o_i,t} | i = 1, \dots, |\mathcal{O}|\}$, and $r_{o_i,t} = R(\mathbf{x}_j, o_i)$.

The reward per action is measured as a normalized difference between the quality of the solutions of the state undertaken, $\mathbf{x}_j$, and the state generated with the selected operator,

$\mathbf{x}'_j = o_i(\mathbf{x}_j)$, where the reward is defined/calculated as follows:

$$F(\mathbf{x}_j) = \frac{\Delta f \times f^*}{f(\mathbf{x}'_j)} \tag{5}$$

where f^* is the globally known best fitness, $f(\mathbf{x}'_j)$ and $f(\mathbf{x}_j)$ are the quality of the solutions of $\mathbf{x}'_j$ and $\mathbf{x}_j$, respectively and $\Delta f = f(\mathbf{x}'_j) - f(\mathbf{x}_j)$. The normalized difference can be negative or positive with respect to the level of improvement.

3.1. Problem state representation and feature analysis

A binary approach is borrowed from [6] to represent the problem states with an input vector of binary variables. Although binary representation is preferable for generalization purposes, it remains not scalable with respect to the size of the problems. In order to overcome this issue, further research has been conducted to identify the representative information captured from fitness landscape and other relevant population-wise information.

Fitness landscape analysis has been studied over decades to help identify the properties of the search (solution) space that can lead to a highly effective search. In particular, the key details of the locality of the solutions can be derived from the fitness landscape analysis. Over the past few decades, a significantly sound literature has been developed for utilizing the most representative information [35–37].

The literature has yielded a number of features that can be used to express the problem states in a more scalable manner. This supports to avoid emerging scalability issue, that was discussed previously. It offers options to use of the experiences gained when solving a range of problems of different sizes and types. Let $\Phi = \{\phi_i | i = 1 \ldots |\Phi|\}$ represent the set of features that can be used in problem characterization, where $|\Phi|$ is the size of the feature set, which in this study is considered to be 19.

These features are partially taken from the properties of the population of solutions and partially extracted from the properties of the individual solutions. The list of metrics, which are to-be-considered as features, extracted from the population properties is tabulated in Table 1, whereas the list extracted from individual properties is tabulated in Table 2. The Appendix contains a list of Equations (A1)–(A19), which represent the corresponding calculation details.

The features listed in Table 1 are population-based metrics. In this study, the Artificial Bee Colony Algorithm (ABC) was chosen as the swarm intelligence framework. The

Table 1. Population-based features.

Feature	Formula
Population Solution Diversity (ϕ_1)	Equation (A1)
Population Fitness Deviation (ϕ_2)	Equation (A2)
Population of new best children (ϕ_3)	Equation (A3)
Proportion of new improving children (ϕ_4)	Equation (A4)
Proportion of amount of improvements (ϕ_5)	Equation (A5)
Proportion of convergence velocity (ϕ_6)	Equation (A6)
Proportion of convergence reliability (ϕ_7)	Equation (A7)
Evolutionary ability of population (ϕ_8)	Equation (A8)
Evolvability of population (ϕ_9)	Equation (A9)
Proportion of average trial number (ϕ_{10})	Equation (A10)
The diameter of population (ϕ_{11})	Equation (A11)

Table 2. Individual solution-based features.

Feature	Formula
Distance between g_{best} and parent solutions (ϕ_{12})	Equation (A12)
Distance between parent and child solutions (ϕ_{13})	Equation (A13)
Fitness gap between g_{best} and child solutions (ϕ_{14})	Equation (A14)
Fitness gap between the parent and the offspring (ϕ_{15})	Equation (A15)
Distance between p_{best} and parent solutions (ϕ_{16})	Equation (A16)
Distance between p_{worst} and parent solutions (ϕ17)	Equation (A17)
Proportion of trial number (ϕ_{18})	Equation (A18)
The success of operator i (ϕ_{19})	Equation (A19)

first 5 metrics, $\{\phi_1, .., \phi_5\}$, were obtained from [38] and implemented for (i.e. tailored to) the ABC. The ABC is one of the very recently developed highly reputed swarm intelligence algorithms [39]. Hamming distance has been used to binarize the metrics that were calculated using the distance measure. This has been investigated by Erwin and Engelbrecht [12] in order to adjust them to binary problem solving approach. The metrics, $\{\phi_6, .., \phi_9\}$ are introduced and proposed by Wang et al. [13] with sound demonstration, while ϕ_{10} is obtained from the trail index used in ABC and utilized to measure/observe the iteration-wise hardness in problem solving. In addition, ϕ_{11} has been used from [40] to calculate the distance between the two farthest individuals within a population/swarm. According to Erwin and Engelbrecht [12], one of the crucial factors in swarms/populations that can help characterize the states is the diversity of populations, whereas [13] discusses the evolvability of populations in environments with dynamic landscape structure.

On the other hand, a number of metrics, or features, can be obtained from the auxiliary information of each individual solution. These characteristics appear to be useful in elements that are specific to each individual and allow the operators to take meaningful action in specific situations.

The individual-related features are tabulated in Table 2. The majority of these features were proposed by Teng et al. [38] with the exception of ϕ_{18}, which is introduced for the first time in this study. Among these features, the success rate for operator i is calculated with $osr_i = \frac{sc_i}{tc_i}$, labelled as ϕ_{19}, where sr_i is the success counter and tc_i is the total usage counter for the operator i.

There are more metrics reported in the literature that are calculated through local search procedures [35, 41]. The set of features listed or tabulated in this article is restricted due to the scope of the study, leaving out many others. To characterize the search space and describe problem states in order to achieve the mapping of problem states to operators, these elicited metrics have only previously been employed separately or in different groups. Several supervised machine learning algorithms have been used in previous work to investigate the impact of this set of features [42]. That investigation gave out a lot of positive results regarding the utilization of the same feature set for capturing instant changes in online search and identifying relevancies to build up experience and apply it for instant decision-making. Given that a generic problem state is denoted by $\mathbf{x}_t$ at iteration t, each problem state subject to a decision will be generated by $\Phi_t = g(\mathbf{x}_t)$, where $\Phi_t = \{\phi_{t,i} | i = 1, \ldots, |\Phi_t|, t = 1, \ldots, T\}$, $g(\mathbf{x}_t)$ is a function of $\mathbf{x}_t$ that runs all Equations (A1)–(A19) to obtain

Φ_t vector, and finally, $Q(\Phi_t, o_a) = \|\Phi_t - c_a\|$. Therefore,

$$\sigma(\Phi_t, F(\mathbf{x}_t)) = \begin{cases} \arg\max_{o_a \in \mathcal{O}} Q(\Phi_t, o_a) & \text{if } rnd \geq \epsilon \\ \text{random}(o_i \in \mathcal{O}) & \text{otherwise} \end{cases} \tag{6}$$

Equation (3) will be implemented with Equation (7) in order to update the cluster centre corresponding to operator $o_a \in \mathcal{O}$ as the operator selected.

$$c_{a,i} = \frac{c_{a,i} + \phi_{t,i}}{n_a} \quad \forall i \in \{1, \ldots, |\Phi_t|\} \tag{7}$$

where $c_{a,i}$ is the ith dimension of the cluster centre representing operator o_a, a is the operator index, n_a is the number of successful use of the operator o_a, t is the iteration index and i is the dimension index for the state data of Φ_t.

3.2. The search space c haracterization

It is well-know that the level of difficulty eventually tightens through progressing search, and problem solving is a set of actions taken to find better and best fitting solutions. This indicates that moving to a better fitting solution is very difficult in the later stages but highly likely in the earlier stages. Obviously, this has an impact on how well operators perform, as any operator can contribute to fitness improvements in the early stages but only the fittest operators can help improve the quality of the solution. Due to this fact, the search space $\mathbb{S}$ is decided to be splitted into a number of sub-spaces, $S_i \in \mathbb{S}$, where $\mathbb{S} = \{S_i | i = 1, .., M\}$ and M is the number of sub-spaces.

The characteristics of each search space will impose different behaviours for each operator, hence, it is expected that the performance of the operators would be affected accordingly. Because of this, it is necessary to redefine how operators are used so that the experiences obtained in each sub-space can be taken into account, recorded independently, and utilized effectively. The set of operators defined as $\mathcal{O} = \{o_a | a = 1, .., |\mathcal{O}|\}$ will be redefined as $\mathcal{O} = \{o_{a,i} | a = 1, \ldots, |\mathcal{O}|, i = 1, \ldots, M\}$. In order to avoid confusions and congestion of indexes, a proxy set of operators, O, is defined as $\mathrm{O} = \{\mathrm{o}_k | k = 1, \ldots, K\}$, where $K = |\mathcal{O}| \times M$. Subsequently, Q values will be $Q(\Phi_t, \mathrm{o}_k) = \|\Phi_t - \mathrm{c}_k\|$ and Equations (6) and (7) will be revised as follows:

$$\sigma(\Phi_t, F(\mathbf{x}_t)) = \begin{cases} \arg\max_{\mathrm{o}_k \in \mathrm{O}} Q(\Phi_t, \mathrm{o}_k) & \text{if } rnd \geq \epsilon \\ \text{random}(\mathrm{o}_k \in \mathrm{O}) & \text{otherwise} \end{cases} \tag{8}$$

$$\mathrm{c}_{k,i} = \frac{\mathrm{c}_{k,i} + \phi_{t,i}}{\mathrm{n}_k} \quad \forall i \in \{1, \ldots, |\Phi_t|\} \tag{9}$$

where $\mathrm{c}_{k,i}$ is the ith dimension of the cluster centre representing operator k, n_k is the number of successful use of the operator k, t is the iteration index and i is the dimension index for the state data of Φ_t.

3.3. Transfer learning using reinforcement learning

Transfer learning is a sub-field of machine learning where the learned experience is formulated for use in processing unseen data and information and solving new problems. Data

modelling is the process of instrumenting algorithms to device machinery that processes unseen data and projects prospective cases. The goal of transfer learning is to increase the effectiveness of the algorithms by processing and translating the experience obtained during previous runs and episodes while solving one or more problems into a transferable form. In this study, transfer learning has been used to apply experience gained through RL to a range of problem instances, both large and small and of various types.

Transfer learning via RL is a relatively unexplored area of research. This approach looks at how knowledge and experience gained through online learning can be transferred into a format that can be used to solve similar problems. Although it is well known that a trained data model can be applied to similar cases with unseen data and the performance can be significantly improved [43], it is not clear how this can be utilized when the problem or domain is entirely different and the interrelation of the components are also different. Traditional machine learning and deep learning algorithms have up until now been designed to work in isolation [35, 41]. These algorithms have been designed to solve specific problems. Once the feature-space distribution changes, the models must be rebuilt from scratch.

Transfer learning turned to be very popular in machine learning that are proposed to overcome the isolated learning paradigm. It enables use of gained knowledge and experience over the previous times while solving other problems previously. The idea is to transfer the gained experience from solving one problem to the other related problems. The transfer learning process is required to address the following three critical questions: what should be transferred, when should it be transferred, and how should it be transferred? The literature includes a number of transfer learning approaches that can be used, depending on the domain, the problem at hand, and the data availability [44]. It is essential to identify the similarities and differences between the problems, which happened to be the problem with which the knowledge to be used gained through in order to solve the problem at the hand. The characteristics of such problems play crucial role in success of the knowledge transferred to be helpful in problem solving undergoing. This turns to gain more importance when RL agents are used in transfer learning. The majority of transfer learning research has been focused on general classical RL problems; however, the proposed work will focus on how to gain transferable experience in *operator selection* through reinforcement learning.

The experience gained can be transferred via a data model defined as $\mathcal{M} : \mathbb{R}^{|D|} \rightarrow \mathbb{R}^{N}$, where D is the data set in hand, $d \in D$ is a piece of data, N is the number of outputs of the model, and the instantiated model is defined as an accumulated structure trained through:

$$\mathcal{M}(D) = \bigcup_{\substack{i \in |\mathcal{M}| \\ d \in D}}^{\infty} \lambda_i m_i(d) \tag{10}$$

It is known that the instantiated data model, $\mathcal{M}(D)$, is a union of training episodes, $m_i(d)$, which is an episodic implementation of $\mathcal{M}(D)$, and is discounted/regulated by the coefficient λ_i. Each upcoming training episode $m_i d$ is aggregated to the model $m(d)$ normalized with λ_i. The aggregations are anticipated to increase the learning agent's degree of experience, subject to a decision over whether to grant the agent permission to accumulate experience in a single run or in an episode.

Equation (10) rule represents the longer run of transfer learning logic. However, this model require to be implemented or instantiated in order to be simplified and implemented as $m(d) = (1 - \delta)\alpha(d) + \delta\beta(d)$, where $\alpha(d)$ and $\beta(d)$ represent two successive

training episodes, $m_i(d)$ and $m_{i+1}(d)$, respectively. While the learning coefficients $\lambda_{i+1} = \delta$ and $\lambda_i = (1 - \delta)$ control the contribution of previous and next experiences, respectively. For example, it switches training ON if $\delta > 0$ and OFF otherwise.

The above-mentioned method for incorporating transfer learning into problem solving employs the data model as an ongoing clustering approach, where the model $\mathcal{M}(D) \leftarrow \mathcal{C}(D)$ is instantiated and trained with solving a specific instance of the problem that includes the following components: $m_i(d) \leftarrow \mathbf{c}_a(d)$, where α represents the learned components, β is the change that will be brought about by impending actions, and δ is the determination of whether or not past experience will be used.

The ABC algorithm is implemented with a learning operator selection scheme, and it is designed to run once to solve a particular problem instance. Online learning is turned ON to train and optimize the cluster centres that represent the operators, and it is turned OFF to repeat the experiments using the same problem instance but different random number sequences. Due to the pruning of the exploration activities, it is anticipated that the problem will be solved in a much shorter amount of time with similar or slightly improved solution quality. This stage, which is still the most popular way of transfer learning, is where the proposed algorithm transfers previously gained experience.

The transfer learning implemented in this study is aimed to be employed in three layers of transfer experience: (i) The agent is trained once by solving a specific benchmark instance of one of the combinatorial problems (such as OneMax or the Set Union Knapsack Problem), and then using that experience to solve several other instances of the same size; (ii) using the trained agent in solving various other benchmark instances of the same problem type that are of different size from the original instance; and (iii) using the trained agent to solve the instance of the other combinatorial problem being studied in this paper. The preliminary results of the layer (i) of this approach with limited depth have been reported in [45]. In this article, we provide further details of the results with relevant discussions in the following section.

4. Experimental results

This section presents the experimental results and evaluates the performance of the proposed algorithm which is coded using Python 3.10 programming language. All the experiments were conducted on the compute server provided by TUBITAK ULAKBIM, High Performance and Grid Computing Center. Two well-known combinatorial optimization problems, which are discussed in the following subsection, have been solved using the proposed approaches.

4.1. Benchmark problems

The algorithms proposed are implemented to demonstrate how the proposed approaches work and offer better solutions. For that purpose, two renowned combinatorial optimization problems have been selected and a number of benchmarking instances have been generated and used for the demonstration accordingly. The selected two problems are *OneMax* and *Set Union Knapsack problems*, which are widely acknowledged and well known in various respects.

Table 3. OneMax problem instances.

Problem ID	1	2	3	4	5	6	7	8	9	10
Dimension	500	750	1000	1250	1500	1750	2000	2250	2500	2750
Problem ID	11	12	13	14	15	16	17	18	19	
Dimension	3000	3250	3500	3750	4000	4250	4500	4750	5000	

OneMax problem is a simple binary problem, which mainly looks for the highest value that can be gained from a set of strings. Let $\mathcal{B}$ be the set of binary values and $b_i \in \mathcal{B}$ is a particular binary value. The main aim is to maximize $\sum_{i=1}^{|\mathcal{B}|} b_i$. The problem has been used to demonstrate the performance of a genetic algorithm by Khair et al. [46]. A number of benchmarking problem instances have been generated with a range of dimensions from 500 to 5000 as tabulated in Table 3.

Set Union Knapsack problem (*SUKP*) is another combinatorial optimization problem, which is usually solved as binary optimization problem. It is known as one of the NP-Hard problems [47], which is attempted by many metaheuristic approaches [48–50]. It requires a set of items to be optimally composed in subsets so as to gain the maximum benefit. Given a set of n elements, $U = \{u_i | i = 1, \ldots, n\}$ with a non-negative weight set, $W = \{w_i | i = 1, \ldots, n\}$ and a set of m items, $\mathcal{K} = \{U_j | j = 1, \ldots, m\}$ with a profit set, $\mathcal{P} = \{p_j > 0 | j = 1, \ldots, m\}$, a subset of $L \subseteq \mathcal{K}$ is sought to be found such that it maximizes the profit subject to the sum of the weights of the selected items not exceeding the capacity constraint, C.

The problem is originally represented in real numbers and needs to be represented in binary form to enable binary operators in search algorithms such as binary ABC [30]. Following the details of the problem and the approach introduced by Wu and He [50], a binary vector, $B = \{b_j | j = 1, .., m\} \in \{0, 1\}$, is defined to be used as the set of decision variables, where $b_j = 1$ if an item is selected, $b_j = 0$, otherwise. The model of the problem can be reformulated as follows:

$$\max \quad f(B) = \sum_{j=1}^{m} b_j p_j \tag{11}$$

$$\text{s.t.} \quad W(L_B) = \sum_{i \in \bigcup_{j \in L_B} U_j} w_i \leq C \tag{12}$$

The main goal is to find the best binary vector, B, which provides the subset of items with the maximum profit.

The problem instances of *SUKP* chosen in this study are collected from recently published literature. In [49], the authors have introduced 30 benchmarking problem instances of *SUKP* as tabulated in Table 4 with all configuration details, where there are 3 different configurations that vary depending on the comparative status of m and n: (i) $m > n$, (ii) $m < n$, and (iii) $m = n$, while $\mathbf{w} \in \{0.10, 0.15\}$ and $\mathbf{y} \in \{0.75, 0.85\}$ represent the density of elements and the rate between the capacities and the sum of weights of elements, respectively. As shown, each set of problem instances includes 10 instances that vary in $m, n, \mathbf{w}$ and $\mathbf{y}$ values.

Table 4. The *SUKP* benchmark instances introduced in the literature [49].

Set_1					Set_2					Set_3				
ID	*m*	*n*	*w*	*y*	ID	*m*	*n*	**w**	**y**	ID	*m*	*n*	**w**	**y**
1_1	100	85	0.10	0.75	2_1	100	100	0.10	0.75	3_1	85	100	0.10	0.75
1_2	100	85	0.15	0.85	2_2	100	100	0.15	0.85	3_2	85	100	0.15	0.85
1_3	200	185	0.10	0.75	2_3	200	200	0.10	0.75	3_3	185	200	0.10	0.75
1_4	200	185	0.15	0.85	2_4	200	200	0.15	0.85	3_4	185	200	0.15	0.85
1_5	300	285	0.10	0.75	2_5	300	300	0.10	0.75	3_5	285	300	0.10	0.75
1_6	300	285	0.15	0.85	2_6	300	300	0.15	0.85	3_6	285	300	0.15	0.85
1_7	400	385	0.10	0.75	2_7	400	400	0.10	0.75	3_7	385	400	0.10	0.75
1_8	400	385	0.15	0.85	2_8	400	400	0.15	0.85	3_8	385	400	0.15	0.85
1_9	500	485	0.10	0.75	2_9	500	500	0.10	0.75	3_9	485	500	0.10	0.75
1_10	500	485	0.15	0.85	2_10	500	500	0.15	0.85	3_10	485	500	0.15	0.85

The Artificial Bee Colony (ABC) algorithm considered in this study has been implemented to embed pool of operators with an operator selection scheme. As explained before, the selection scheme is developed and empowered with reinforcement learning so as to automatize the whole mechanism. The ABC framework has three main parameters to setup; maximum iteration, the number of solutions in the population, and the number of maximum trials (limit). For the *OneMax* problem, the swarm/population size is set to $N = 20$ and the maximum iteration is fixed at 250, whereas for *SUKP* cases, the swarm/population size is determined by the dimensionality of the problem instances, with $max\{m, n\}$, where m is the number of items and n is the number of elements in an instance.

The pool of operators consists of four binary operators, which have recently been used in the literature, $\mathcal{O} = \{flip\text{ABC}, n\text{ABC}, ibin\text{ABC}, nb\text{ABC}\}$. *flip*ABC simply inverts a randomly selected bit, while the others – *n*ABC, *ibin*ABC and *nb*ABC – are more complex-structured operators taken from [51, 52] and [53], respectively. According to the conditions of the search algorithm, an operator selection scheme is aimed to function for selecting the most productive operator from this pool.

Four variants of the Reinforcement Learning (RL)-based adaptive operator selection schemes embedded in ABC have been tested. The first variant is labelled with *One-Run* that disregards previous learning experiences and starts learning from scratch in each repetition re-initializing all parameters of the algorithm. On the other hand, the variant labelled with *All-Run* does not dispose gained experience up-to-date and transfers all relevant operator data to the next runs. The variant labelled as *One-Run w/L* has uploaded information previously recorded on the specific problem instance (for the *OneMax* problem 2500 dimensions, for the *SUKP* ID X) after 30 runs. The variant labelled as *All-Run w/L* has the same idea but only upload the information for the first run, because the information is transferred to the next run.

4.2. Results and discussions

The experimental results presented in the following sections was conducted to evaluate the efficiency of problem representation using the set of features introduced in Section 3.1, splitting search space approach as in Section 3.2, and using the transfer learning detailed in Section 3.3. The efficiency gained by splitting the search space was first investigated since the rest of the experiment is based on those results.

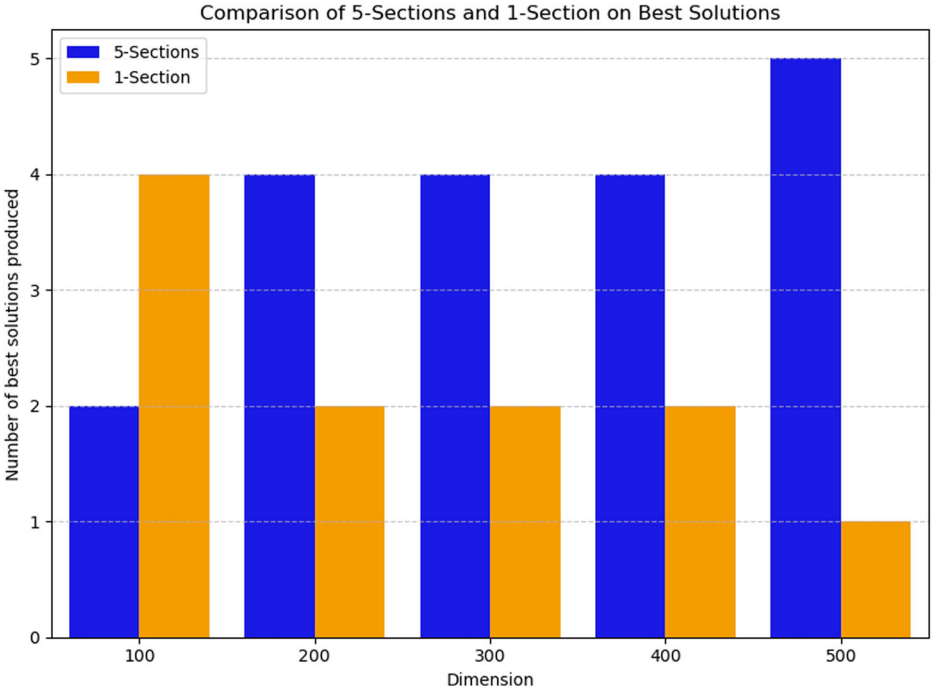


Figure 2. Experimental results for splitting search space into pieces and treating each as an independent space.

4.2.1. Splitting search space

The search space of a problem may hold different characteristics in different regions since operators' behaviours can vary region-to-region. Therefore, in order to enable the operator selection scheme to learn how to select the operators according to different problem states and stages of evolution, we decided to divide the search space into a number of sub-spaces. The new pool of operators turns to be $\mathrm{O} = \{flip\mathrm{ABC}_i, n\mathrm{ABC}_i, ibin\mathrm{ABC}_i, nb\mathrm{ABC}_i | i = 1, \ldots, 5\}$, as an extension of $\mathcal{O}$. Here, the same pool of operators has been applied to each sub-space separately without reusing any experience data gained from previous (other) sub-spaces. Therefore, the adaptive operator selection scheme is trained to learn when to select each operator subject to the circumstances of each sub-space.

Figure 2 present comparative results in bar chart considering two options; the complete search space labelled with *1-section*, and a split of 5 sub-spaces labelled with *5-section*. On the x-axis, the dimensions of *OneMax* problem instances plotted against the quality of the solution on *y*-axis. Each *x*-point represents 6 different instances in the same dimension, where the quality of the solution is attained much better in higher dimension with 5-section splits noting that 1-section approach achieves higher than 5-section in 100-dimension problems only. This clearly demonstrates the fact that splitting search space helps to achieve better performance.

4.2.2. Feature-based problem representation

The problem has been represented with a set of 19 features as introduced earlier. The idea is to demonstrate that a good quality of learning can be achieved with a set of latent features

– alculated accordingly – *n* order to make our proposed approach more scalable compared to the binary representation suggested by Durgut et al. [6]. The approach in [6] is strictly specific to the problem size, and the results are encouraging but not scalable. In order to overcome these limitations, we proposed in this paper a feature-based representation to avoid the restrictions imposed by the size of the problems.

The experimentation in this section is conducted to make sure there is no difference between the performances of ABC algorithms implemented binary representation and ABC variants with feature-based representation. The key hypothesis turns to be that "*there is no difference in the performance of the algorithms with either representation approaches*". For testing this hypothesis, we will change the problem representation only and no other structure or parameter of the algorithms. Once we hold this hypothesis, this will mean that the restriction imposed by the size of the problem instances is avoided and the approach will be scalable.

Here, the set of features used in representations is presented in Section 3.1 and tabulated in Tables 1 and 2. It worthwhile to note that all ABC variants implemented either representation approach are furnished with a set of binary operators selected with an adaptive selection scheme based on reinforcement learning.

In order to conduct hypothesis testing, RLABC algorithm – as an ABC variant – is developed and furnished with feature-based representation, a set of binary operators and an adaptive selection scheme based on Q learning as explained before. It is to solve *SUKP* problem instances as it was described to solve *OneRun* problems, too.

Figure 3 presents comparative results of *OneRun* as the proposed approach – explained above – and RLABC-TC algorithm collected from [45]. The plots demonstrate that there is clearly no significant difference between the results by both algorithms. The *SUKP* problem instances under consideration in this experimentation are introduced in Table 4, where they can be grouped into 3 with respect to their sizes. The *x*-axis of Figure 3 presents the labels of the instances while *y*-axis is the quality of solution measured as the mean of 30 repetitions. The box-plots of the results by each algorithm scattered on the figure, where the quality

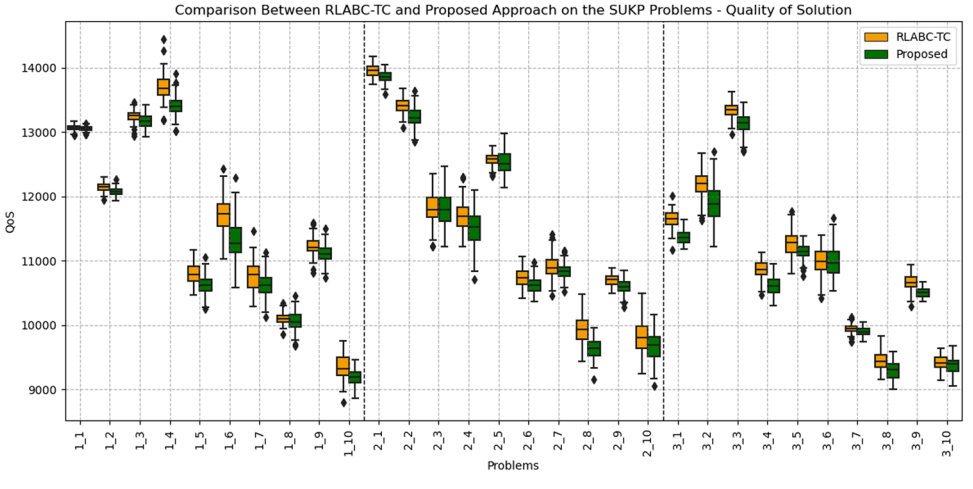
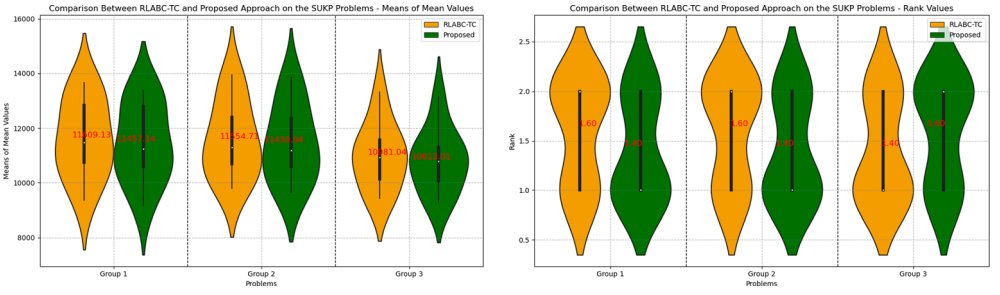


Figure 3. Quality of solutions comparison on the SUKP problems between RLABC-TC and One Run (Proposed) approaches.



(a) Means Comparison on the *SUKP* Problems (b) Ranks Comparison on the *SUKP* Problems

Figure 4. Comparative results in means and ranks produced by RLABC-TC and *OneRun* (Proposed) approaches for *SUKP* problems between, (a) means comparison on the *SUKP* problems, (b) ranks comparison on the *SUKP* problems.

of solutions in mean look almost overlapping – with non-significant (slight) differences – which suggests that the hypothesis of no difference is held. This is supported by Figure 4(a) plotting the violins of the means of means per group of *SUKP* instances in the same size and Figure 4(b) plotting the violins of the averaged ranks per group in the same size calculated with Wilcoxon signed-rank test. Both plots confirm that there is no significant differences in between the quality of solution results of the compared algorithms. Particularly, The violins in Figure 4(a) representing each algorithm per group look so much alike, which confirms no significant difference, while The violins in Figure 4(b) look comparable and illustrate slight difference.

4.2.3. Utilizing transfer learning

The remaining part of the proposed work involves training the RL agents for new problems and problem instances through transferring gained experiences. It is aimed to preserve information gained through solving problems with a given type, size, and setting so that it may be utilized in solving problems of a new type, size, complexity, and settings. For this reason, two different variants of RL-agents are created; the variants that use past experience are labelled as *w/L*, and the variants that do not use past experiences but instead begin learning from scratch each time are labelled as *w-L*. These two new additional variants are added to the two main variants, *One-Run* and *All-Run*, as previously mentioned, to provide a total of four variants all together.

Table 5 presents the comparative results of the four variants of RL-based ABC – as introduced previously – to solving the *OneMax* benchmarks with respect to 'Rank', 'Max', 'Mean', 'Std', and additionally the 'p-values' from the statistical test. It can be observed that the variants that carrying over the previous experience help in producing better solutions compared to those starting from the scratch. The mean of Wilcoxon ranks of *One-Run w-L* and *One-Run w/L* are 3.21 and 1.94, respectively. In addition, the latter variant seems performed better in terms of 'Max' values with score of 14 out of 20 instances. Similarly, *All-Run w/L* improves the performance compared to *All-Run w-L* with the mean ranks of 1.94 and 2.89, respectively. Apart from the first 6 ones, the majority of the results are the best for all of the benchmark instances. As the performance of many approaches, including those used in this study, remain quite near to one another, this suggests that these instances are highly competitive.

Table 5. Comparative results using the variant algorithms under investigation for the *OneMax* problem instances.

	One-Run w-L				One-Run w/L					All-Run w-L				All-Run w/L				
ID	Rank	Max	Mean	Std	Rank	Max	Mean	Std	*p*	Rank	Max	Mean	Std	Rank	Max	Mean	Std	*p*
1	3	500	499.50	0.81	2	500	499.53	0.67	9.1E-01	1	500	499.80	0.40	4	500	499.27	0.81	1.1E-02
2	1	749	744.77	2.29	2	749	744.70	3.08	7.2E-01	3	748	744.67	2.10	4	749	743.63	2.61	7.7E-02
3	2	989	984.03	3.38	1	989	984.87	2.78	2.6E-01	3	990	983.60	3.48	4	992	981.40	4.22	4.0E-02
4	3	1224	1214.87	5.73	1	1228	1216.77	5.49	1.5E-01	2	1224	1216.57	4.99	4	1223	1209.17	6.16	3.7E-04
5	3	1448	1437.20	7.18	2	1453	1437.70	7.52	9.6E-01	1	1454	1438.60	8.65	4	1442	1430.20	6.81	2.3E-04
6	3	1664	1646.37	8.62	1	1663	1649.20	7.57	1.0E-01	2	1668	1646.60	9.61	4	1657	1645.17	6.46	5.4E-01
7	3	1861	1841.97	13.73	2	1862	1843.97	9.83	9.4E-01	4	1873	1841.90	13.34	1	1880	1856.00	11.75	3.6E-04
8	4	2054	2030.77	14.57	2	2066	2037.60	13.22	4.7E-02	3	2058	2031.90	15.81	1	2079	2057.50	9.92	4.1E-06
9	4	2251	2211.33	19.74	2	2258	2223.97	16.98	2.7E-02	3	2246	2218.33	14.97	1	2283	2257.03	12.16	2.6E-06
10	3	2435	2395.33	18.65	2	2449	2400.23	19.38	3.9E-01	4	2445	2395.03	19.87	1	2475	2448.07	12.56	1.7E-06
11	2	2621	2572.20	17.42	3	2601	2572.00	21.10	6.3E-01	4	2608	2563.23	19.30	1	2655	2639.13	10.48	1.7E-06
12	3	2786	2747.77	18.98	2	2790	2752.03	19.54	3.5E-01	4	2801	2743.90	23.20	1	2850	2819.70	15.75	2.1E-06
13	3	2937	2903.00	18.34	2	2954	2923.27	23.73	6.9E-04	4	2943	2902.27	20.55	1	3026	3001.80	13.21	1.7E-06
14	4	3125	3081.50	20.47	2	3151	3092.20	24.86	1.9E-01	3	3157	3083.47	29.91	1	3201	3177.63	15.88	1.7E-06
15	4	3305	3250.47	24.49	2	3321	3263.50	28.86	3.7E-02	3	3293	3251.60	29.76	1	3383	3353.53	16.52	1.7E-06
16	4	3462	3404.60	25.20	2	3487	3421.37	36.35	1.3E-02	3	3491	3418.50	31.27	1	3573	3531.77	17.03	1.7E-06
17	4	3642	3563.90	34.78	2	3647	3585.27	39.62	2.1E-02	3	3621	3577.47	24.08	1	3761	3707.33	19.51	1.7E-06
18	4	3797	3739.50	28.98	2	3797	3750.53	33.00	2.7E-01	3	3813	3750.00	33.75	1	3918	3880.30	19.53	1.7E-06
19	4	3951	3904.90	34.57	3	3980	3909.50	41.29	6.8E-01	2	3965	3911.13	25.10	1	4078	4043.13	16.73	1.7E-06
	3.21				1.94					2.89				1.94				

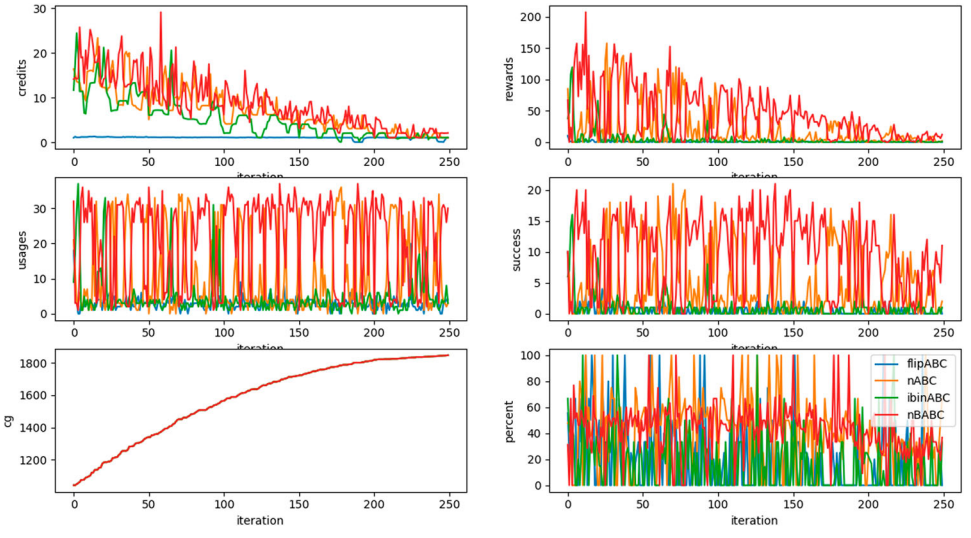


Figure 5. The operator information graphs of the *OneMax* problem.

Figure 5 depicts the performance obtained by the four operators through iterations on a OneMax problem instance with 4000 dimensions over a single run. The figure consists of 6 sub-figures; *top-left* indicates the progression of credit level, *top-right* presents the change in rewards, *mid-left* shows the changes in usage of the operators, *mid-right* displays the changes in the success of the operators, *bottom-left* shows the overall convergence, while *bottom-right* figure presents the selectability of the operators in percentile. All the sub-figures demonstrate the approximation of the behaviours over increasing iterations. This suggests the progression of learning on how to select the operators subject to the circumstances. The search process is split into five regions. It is clearly shown that *n*ABC and *nb*ABC are mostly used operators during the search process. Both received greater rewards than the others, most likely as a result of the update mechanisms of each being better suited to address the *OneMax* problems. It should be noted that *flip*ABC can change only one bit per operation, hence the quality of the solution can maximally increase by 1 only. As a result, this operator is less frequently utilized.

Table 6 outlines the comparative results of the four variants of RL-based ABC in terms of 'Rank', 'Max', 'Mean', 'Std', and additionally the '*p*-values' from the statistical test in solving the *SUKP* benchmark instances. The overall mean ranks of the variants of *One-Run w-L*, *One-Run w/L*, *All-Run w-L*, and *All-Run w/L* are 2.17, 2.43, 2.97 and 2.43, respectively, where the best mean rank is 2.17 achieved by the *One-Run w-L*. *One-Run w/L* and *All-Run w/L* – the variants with transfer learning – produce with an equal performance level with the score of 2.43 in mean rank, which remain slightly worse. In terms of 'Max' column, both *One-Run w-L* and *All-Run w/L* have achieved the best solutions; 14 times out of the 30 instances, while *One-Run w/L* and *All-Run w-L* have produced 13 and 11 times out of 30 instances, respectively.

It is observed that, transfer learning appears to help the *All-Run* variant while having little effect on the *One-Run* variant. Although the work by Durgut et al. [45] demonstrated the benefit of transfer learning with preliminary result and the performance of both *One-Run*

Table 6. Comparative results using the variant algorithms under investigation for the *SUKP* problem instances.

	One-Run w-L				One-Run w/L					All-Run w-L				All-Run w/L				
ID	Rank	Max	Mean	Std	Rank	Max	Mean	Std	*p*	Rank	Max	Mean	Std	Rank	Max	Mean	Std	*p*
1_1	1	13167	13052.20	30.68	2	13167	13046.23	23.49	3.6E-01	4	13044	13034.87	44.27	3	13046	13042.20	7.01	6.8E-01
1_2	1	12274	12076.95	65.72	2	12235	12023.00	122.22	1.4E-01	3	12130	11993.63	168.86	4	12274	11979.93	181.49	9.3E-01
1_3	1	13340	13165.27	106.95	4	13340	13107.37	117.23	1.5E-01	3	13319	13126.80	109.97	2	13402	13148.77	116.7	3.9E-01
1_4	1	13671	13376.13	191.36	3	13671	13319.03	210.72	2.5E-01	4	13671	13297.73	205.28	2	13660	13319.70	177.16	7.0E-01
1_5	1	10980	10588.03	160.02	4	10968	10537.80	152.81	1.4E-01	3	10896	10548.63	132.44	2	10972	10549.00	200.13	9.3E-01
1_6	4	12012	11355.87	308.24	1	12135	11502.87	318.34	4.3E-02	3	12035	11383.47	278.75	2	12135	11444.43	330.27	3.8E-01
1_7	1	10994	10616.67	180.01	2	11001	10594.87	216.73	6.6E-01	3	11244	10580.90	231.11	4	10944	10537.27	206.51	5.2E-01
1_8	1	10168	10055.10	137.99	4	10168	10032.90	168.48	5.2E-01	2	10182	10036.23	139.19	3	10175	10034.13	149.66	8.7E-01
1_9	2	11426	11118.60	150.97	1	11296	11120.30	129.08	9.7E-01	4	11296	11042.73	127.75	3	11296	11054.93	130.83	9.8E-01
1_10	1	9409	9166.57	127.46	4	9486	9105.27	138.28	4.8E-02	3	9486	9140.33	159.16	2	9494	9144.17	201.14	9.8E-01
2_1	3	14044	13850.57	79.48	1	13963	13852.63	59.98	8.8E-01	4	13963	13843.27	68.49	2	14044	13850.63	75.75	7.6E-01
2_2	4	13407	13222.67	149.27	1	13498	13297.03	162.32	8.9E-02	2	13407	13278.13	120.32	3	13498	13257.77	179.72	4.7E-01
2_3	1	12350	11792.00	253.90	3	12328	11774.40	230.48	4.7E-01	4	12242	11708.17	235.69	2	12103	11780.77	228.71	3.7E-01
2_4	3	11821	11550.13	257.17	2	11846	11592.20	211.66	4.7E-01	1	11830	11613.43	192.09	4	12187	11545.47	252.33	2.5E-01
2_5	1	12644	12523.10	166.57	3	12644	12462.20	167.07	1.4E-01	4	12644	12417.17	228.26	2	12644	12484.97	187.49	2.2E-01
2_6	1	10735	10618.93	118.84	4	10666	10558.13	201.66	1.9E-01	2	10731	10571.00	186.18	3	10666	10569.10	157.88	8.8E-01
2_7	3	11001	10832.70	112.07	1	11282	10870.67	152.82	3.4E-01	4	11160	10831.37	135.25	2	11110	10834.43	141.60	9.4E-01
2_8	3	10089	9665.67	170.46	1	10355	9720.03	222.45	3.4E-01	4	10250	9600.00	208.11	2	10355	9666.77	213.64	4.5E-01
2_9	2	10734	10592.90	94.68	3	10735	10587.90	88.78	9.8E-01	4	10784	10585.33	94.82	1	10902	10603.00	100.29	5.3E-01
2_10	2	10176	9651.73	219.47	1	10176	9720.33	199.58	9.6E-01	3	10176	9592.83	172.32	4	9977	9588.27	188.11	5.4E-01
3_1	4	11579	11348.37	96.03	2	12020	11392.00	181.24	3.8E-01	1	12045	11409.47	201.67	3	11752	11386.73	120.57	7.9E-01
3_2	4	12369	11864.07	289.56	3	12369	11917.87	282.67	4.7E-01	2	12369	11923.83	345.64	1	12369	11950.50	288.43	7.5E-01
3_3	1	13392	13141.20	174.84	4	13500	13118.67	185.12	7.8E-01	3	13477	13119.73	211.00	2	13458	13132.27	225.99	7.5E-01
3_4	4	10920	10620.63	138.02	1	10920	10675.17	161.28	2.3E-01	3	10920	10623.07	118.08	2	10920	10642.53	136.11	6.1E-01
3_5	1	11538	11154.93	147.54	3	11538	11099.40	129.45	1.4E-01	2	11538	11101.37	172.69	4	11538	11085.17	137.35	8.4E-01
3_6	4	11414	10941.50	218.08	2	11377	10953.23	213.23	6.4E-01	3	11237	10944.23	187.32	1	11377	10980.60	235.73	4.7E-01
3_7	2	10068	9911.37	70.59	3	10026	9898.70	64.92	4.5E-01	4	10294	9898.60	100.12	1	10176	9916.87	82.33	2.5E-01
3_8	2	9588	9294.77	138.33	3	9496	9294.43	119.37	9.6E-01	4	9532	9262.50	139.24	1	9796	9297.50	177.36	5.4E-01
3_9	3	10653	10494.07	84.91	1	10636	10511.57	73.70	3.8E-01	2	10732	10504.37	97.57	4	10636	10482.23	95.90	5.6E-01
3_10	3	9514	9359.17	142.50	4	9561	9299.63	144.89	1.2E-01	1	9514	9380.23	136.25	2	9514	9366.73	88.95	4.0E-01
Mean	2.17				2.43					2.97				2.43				

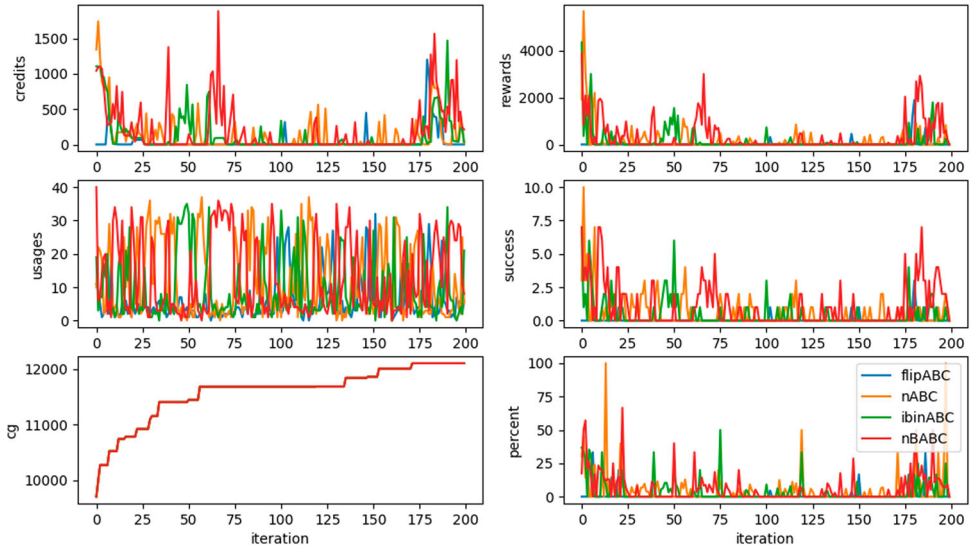


Figure 6. The operator information graph of the *SUKP* problem.

and *All-Run* variants over solving *OneMax* problems supporting this idea, further research and investigation would be necessary to see the real benefit of transfer learning in this regard. It appears that the varied qualities of the implemented operators used to solve the two problems, which differ in structure and properties, contribute to the success of transfer learning in *OneMax* but not in the *SUKP* problems. The issue may be a result of the binary operators being employed being more complementary to one another for one problem type but not for the other. For instance, *flip*ABC operator changes only one bit each time once it has been selected, which may help one problem to rescue from local optima, but not for the other. This leads to the possibility that the operators promoted by learning for *OneMax* might not be as effective for *SUKP*. Obviously, this suggests that further research and investigations are necessary on the underlying RL configuration and the selection of operators in the pool.

Figure 6 represents the convergence behaviour of the variants of *SUKP* problem. This figure has also been plotted in the same way as Figure 5, where 6 sub-figures are included indicating the same set of measures. It is observed that *All-Run w/L* and *One-Run w-L* are the fastest variants on convergence to the optimal solution. Also, the figure shows the performances gained by the four operators through iterations on *SUKP* problem. It is observed that the operator selection is very competitive during the search process at the beginning, (i.e. in the early stages) of the search process in which *n*ABC and *nb*ABC are more dominant, while in the later stages all operators participate in the selection process. A stable non-progressive situation is observed between iterations 75 and 175, which can be clearly seen from all the sub-graphs. The operators are more proactive in the rest of the process over the iterations. This might be due to a special circumstance with the benchmark instances used for this single run. Although the operators did not achieve success between iterations 75 and 175, they eventually turned to escape from local optima, most likely using the *flip*ABC and *n*BABC operators, which appear to be more effective than the other two.

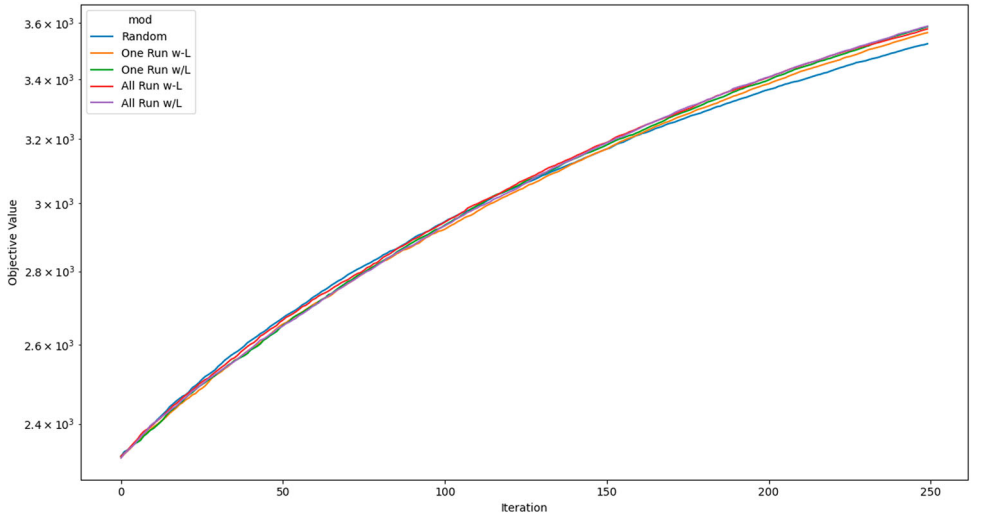


Figure 7. The convergence graph of the *OneMax* problem.

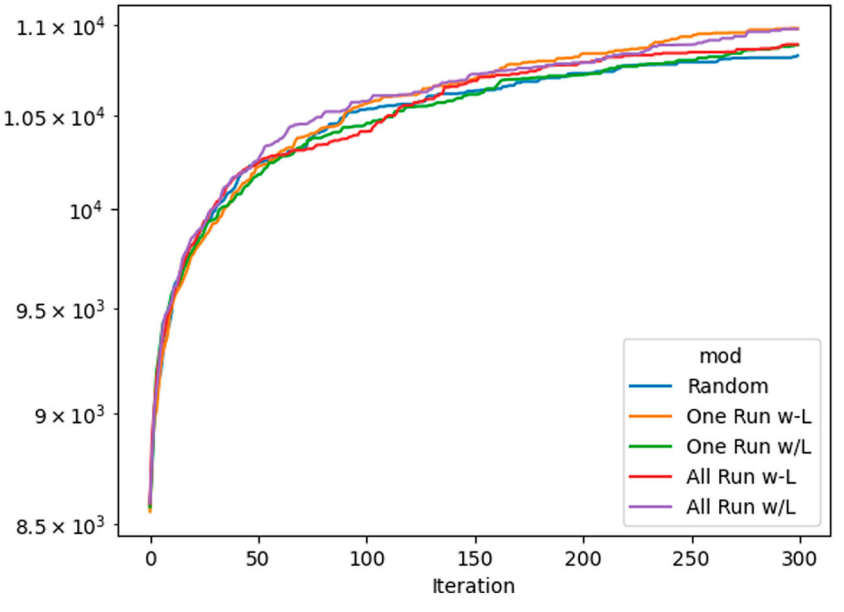


Figure 8. The convergence graph of the *SUKP* problem.

The overall convergence achieved by all the variants of the algorithm for solving both *OneMax* and *SUKP* problems are plotted in Figures 7 and 8. It can be observed that, the plots in both cases initially appear to be highly overlapping, but after 175 iterations in the case of *OneMax* and after 40 in the case of *SKUP*, they begin to diverge. The overall performance depicted appear to be very consistent with the tabulated results. Also, *OneMax* problems are not much complicated compared to the *SUKP* problems.

In order to further demonstrate the performance of the proposed approach for *SUKP*, a comparison analysis is conducted with the state-of-the-art algorithms. Table 7 presents the comparative results, where the proposed algorithm outperforms the others in most of the cases. The algorithms considered for comparisons have been selected from the most relevant literature, where A-SUKP, GA, BABC, ABCBin, and binDE results were captured from [49, 50] and GPSO∗ is from [48], which are published within the last 5 years. Among them, A-SUKP is a greedy local search algorithm while the others are population-based metaheuristic algorithms; GA and binDE are evolutionary algorithms, GPSO∗ is a particle swarm optimization algorithm, and the rest are the variants of binary ABC algorithms, which are the most relevant approaches.

Table 7 tabulates the 'Mean' and 'Max' scores of each approaches for the benchmark instances, while Figure 9 plots the average rankings, which are indicated as 6.83, 4.33, 2.60, 5.87, 4.03, 3.17 and 1.17 for A-SUKP, GA, BABC, ABCBin, bin DE, GPSO∗ and 'Proposed', respectively. The proposed approach is the outperforming variant, *One-Run w-L*, from Table 6. Figure 9 confirms that the best rank among the compared algorithms clearly goes to 'Proposed' approach with the score of 1.17 while the first runner is BABC with 2.60 and the second runner is GPSO∗ with 3.17.

5. Conclusions and future work

This paper presents some results of an ongoing research – rather a journey – in investigating for a generalizable approach for setting up a data-driven operator selection scheme. The main point is to address the critical challenge of balancing exploration and exploitation in optimization process by introducing a novel approach that integrates multi-operator neighbourhood structures with reinforcement learning into the search process. Previously, some steps have been conducted towards a generalization in this regard, but scalability was an outstanding issue and challenge to address. While traditional methods like probability matching and multi-armed bandit techniques offer valuable solutions, they often face difficulties to adapt to dynamic search environments. The proposed framework overcomes these limitations by presenting key contributions: orchestrating multiple operators using RL, employing a feature-based input space to resolve scalability issues, redefining the search space according to optimization process stages, and enabling agents to transfer their learning across various problem instances without size constraints.

This study delivered an in-depth investigation and experimental assessment across three key areas: (i) how to gain experience in search processes to adaptively select operators using RL, (ii) how to develop a scalable approach that allows the operator selection agent to be trained once and applied across diverse circumstances, and (iii) how to transfer the gained experience across different problem instances and types. The results validate the success of the first two objectives, while the third has been partially achieved, indicating a need for further research into more advanced RL models.

The research serves as a proof-of-concept demonstrating RL's potential in developing adaptive operator selection schemes, yet highlights the need for enhancements in learning quality for complex problem types. Additionally, exploring more sophisticated techniques, such as deep learning for improved state-action mapping and multi-agent RL for collective intelligence, can further refine the approach. This study establishes a foundation for future

Table 7. Comparative results considering the state-of-the-art works for the *SUKP* problem.

	A-SUKP		GA		BABC		ABCBin		binDE		GPSO∗		Proposed	
ID	Max	Mean	Max	Mean	Max	Mean	Max	Mean	Max	Mean	Max	Mean	Max	Mean
1_1	12459	12459	13044	12956	13251	13029	13044	12819	13044	12991	13167	12937	13167	13052.20
1_2	11119	11119	12066	11546	12238	12155	12238	12049	12274	12124	12210	11778	12274	12076.95
1_3	11292	11292	13064	12493	13241	13064	12946	11862	13241	12941	13302	12766	13340	13165.27
1_4	12262	12262	13671	12803	13829	13359	13671	12537	13671	13110	13993	12949	13671	13376.13
1_5	8941	8941	10553	9981	10428	9995	9751	9339	10420	9899	10600	10090	10980	10588.03
1_6	9432	9432	11016	10350	12012	10903	10913	9958	11661	10499	11935	10750	12012	11355.87
1_7	9076	9076	10083	9642	10766	10065	9674	9188	10576	9681	10698	9947	10994	10616.67
1_8	8514	8514	9831	9327	9649	9136	8978	8540	9649	9021	10168	9417	10168	10055.10
1_9	9864	9864	11031	10568	10784	10452	10340	9910	10586	10364	11258	10566	11426	11118.60
1_10	8299	8299	9472	8693	9090	8858	8789	8364	9191	8784	9759	8779	9409	9166.57
2_1	13634	13634	14044	13806	13860	13735	13860	13547	13814	13676	13963	13740	14044	13850.57
2_2	11325	11325	13145	12235	13508	13352	13498	13103	13407	13212	13498	12937	13407	13222.67
2_3	10328	10328	11656	10889	11846	11194	11191	10424	11535	10969	11972	11233	12350	11792.00
2_4	9784	9784	11792	10828	11521	10945	11287	10346	11469	10717	12167	11027	11821	11550.13
2_5	10208	10208	12055	11755	12186	11946	11494	10922	12304	11865	12736	11934	12644	12523.10
2_6	9183	9183	10666	10099	10382	9860	9633	9187	10382	9710	10724	9907	10735	10618.93
2_7	9751	9751	10570	10112	10626	10101	10160	9549	10462	9976	11048	10400	11001	10832.70
2_8	8497	8497	9235	8794	9541	9033	9033	8366	9388	8768	10264	9195	10089	9665.67
2_9	9615	9615	10460	10185	10755	10328	10071	9738	10546	10228	10647	10205	10734	10592.90
2_10	7883	7883	9496	8883	9318	9181	9262	9618	9312	9096	9839	9107	10176	9651.73
3_1	10231	10231	11454	11093	11664	11183	11206	10880	11352	11075	11710	11237	11579	11348.37
3_2	10483	10483	12124	11326	12369	12082	12006	11485	12369	11876	12369	11684	12369	11864.07
3_3	11508	11508	12841	12237	13047	12523	12308	11668	13024	12278	13298	12514	13392	13141.20
3_4	8621	8621	10920	10352	10602	10151	10376	9684	10547	10085	10856	10208	10920	10620.63
3_5	9961	9961	10994	10640	11158	10776	10269	9957	11152	10661	11310	10762	11538	11154.93
3_6	9618	9618	11093	10190	10528	9898	10051	9424	10528	9832	11226	10309	11414	10941.50
3_7	8672	8672	9799	9433	10085	9538	9235	8905	9883	9315	8971	9552	10068	9911.37
3_8	8064	8064	9173	8704	9456	9090	8932	8407	9352	8847	9389	8881	9588	9294.77
3_9	9559	9559	10311	9993	10823	10483	10537	9615	10728	10159	10595	10145	10653	10494.07
3_10	8157	8157	9329	8849	9333	9086	8799	8348	9218	8920	9807	8917	9514	9359.17

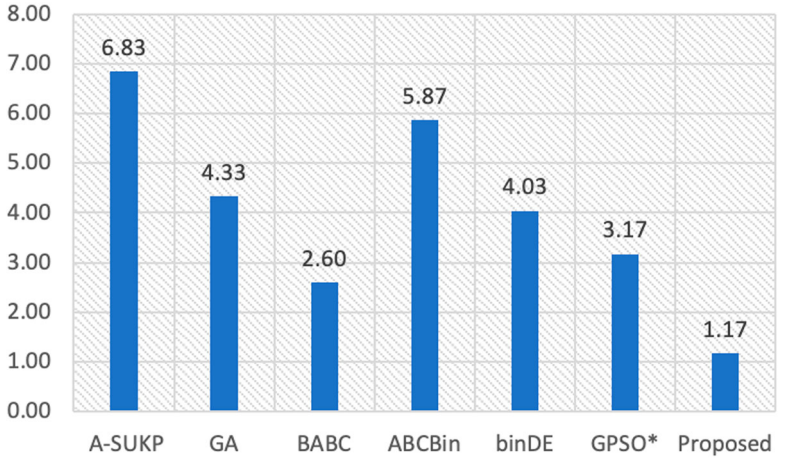


Figure 9. The average ranks calculated over the comparative performances of the state-of-the art approaches to solving the *SUKP* problem instances.

advancements in adaptive optimization by offering a flexible and scalable solution to the operator selection challenge.

Acknowledgements

The numerical calculations reported in this paper were fully/partially performed at TUBITAK ULAKBIM, High Performance and Grid Computing Center (TRUBA resources). The second author was supported by the TUBITAK-2219 program.

Disclosure statement

No potential conflict of interest was reported by the author(s).

Funding

This work was supported by Türkiye Bilimsel ve Teknolojik Araştırma Kurumu [TUBITAK-2219].

References

[1] Chen X, Tianfield H, Li K. Self-adaptive differential artificial bee colony algorithm for global optimization problems. Swarm Evol Comput. 2019;45:70–91. doi: 10.1016/j.swevo.2019.01.003
[2] Belluz J, Gaudesi M, Squillero G, et al. Operator selection using improved dynamic multi-armed bandit. In: Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation; New York, NY, USA. Association for Computing Machinery; 2015. p. 1311–1317; GECCO '15.
[3] Thierens D. An adaptive pursuit strategy for allocating operator probabilities. In: Proceedings of the 7th annual conference on Genetic and evolutionary computation; 2005. p. 1539–1546.
[4] Li K, Fialho A, Kwong S, et al. Adaptive operator selection with bandits for a multiobjective evolutionary algorithm based on decomposition. IEEE Trans Evolut Comput. 2013;18(1):114–130. doi: 10.1109/TEVC.2013.2239648
[5] Scott SL. A modern bayesian look at the multi-armed bandit. Appl Stoch Models Bus Ind. 2010;26(6):639–658. doi: 10.1002/asmb.v26.6

[6] Durgut R, Aydin ME, Atli I. Adaptive operator selection with reinforcement learning. Inf Sci (Ny). 2021;581:773–790. doi: 10.1016/j.ins.2021.10.025
[7] Jeong HY, Song BD. Meta-learning-based adaptive operator selection for traveling salesman problem. Appl Soft Comput. 2025;185:113930.
[8] Ming F, Gong W, Xue B, et al. Automated configuration of evolutionary algorithms via deep reinforcement learning for constrained multiobjective optimization. IEEE Trans Cybern. 2025;55(12):5877–5890. doi: 10.1109/TCYB.2025.3603251
[9] Brambila-Hernández JA, García-Morales MA, Fraire-Huacuja HJ, et al. Novel dynamic decomposition-based multi-objective evolutionary algorithm using reinforcement learning adaptive operator selection (dmoea/d-sl). Comput System. 2024;28(2):739–749.
[10] Yin S, Xiang Z. Adaptive operator selection with dueling deep q-network for evolutionary multi-objective optimization. Neurocomputing. 2024;581:127491. doi: 10.1016/j.neucom.2024.127491
[11] Tianyi Y, Yuren Z. Runtime analysis of adaptive selection variation operators in evolutionary algorithm with reinforcement learning. Appl Intell. 2025;55(12):853. doi: 10.1007/s10489-025-06687-9
[12] Erwin K, Engelbrecht A. Diversity measures for set-based meta-heuristics. In: 2020 7th International Conference on Soft Computing & Machine Intelligence (ISCMI); IEEE; 2020. p. 45–50.
[13] Wang M, Li B, Zhang G, et al. Population evolvability: dynamic fitness landscape analysis for population-based metaheuristic algorithms. IEEE Trans Evolut Comput. 2017;22(4):550–563. doi: 10.1109/TEVC.2017.2744324
[14] Karaboga D, Akay B. A comparative study of artificial bee colony algorithm. Appl Math Comput. 2009;214(1):108–132.
[15] Düğenci M, Aydemir A, Esen İ, et al. Creep modelling of polypropylenes using artificial neural networks trained with bee algorithms. Eng Appl Artif Intell. 2015;45:71–79. doi: 10.1016/j.engappai.2015.06.016
[16] Agarwal D, Gupta A, Singh PK. A systematic review on artificial bee colony optimization technique. Inter J Control Theory Appl. 2016;9(11):5487–5500.
[17] Sutton RS, Barto AG. Reinforcement learning: an introduction. 2nd ed. Cambridge (MA): The MIT Press; 2018.
[18] Davis L. Adapting operator probabilities in genetic algorithms. In: Proceedings of the 3rd International Conference on Genetic Algorithms; 1989. p. 61–69.
[19] Lin Q, Liu Z, Yan Q, et al. Adaptive composite operator selection and parameter control for multi-objective evolutionary algorithm. Inf Sci (Ny). 2016;339:332–352. doi: 10.1016/j.ins.2015.12.022
[20] Verheul J. The influence of using adaptive operator selection in a multiobjective evolutionary algorithm based on decomposition [master's thesis]. Utrecht University; 2020.
[21] Handoko SD, Nguyen DT, Yuan Z, et al. Reinforcement learning for adaptive operator selection in memetic search applied to quadratic assignment problem. In: GECCO Proceedings; New York, NY, USA. Association for Computing Machinery; 2014. p. 193–194; GECCO Comp '14.
[22] Sallam KM, Elsayed SM, Sarker RA, et al. Landscape-based adaptive operator selection mechanism for differential evolution. Inf Sci (Ny). 2017;418:383–404. doi: 10.1016/j.ins.2017.08.028
[23] Consoli PA, Mei Y, Minku LL, et al. Dynamic selection of evolutionary operators based on online learning and fitness landscape analysis. Soft Comput. 2016;20(10):3889–3914. doi: 10.1007/s00500-016-2126-x
[24] Kizilay D, Tasgetiren MF, Oztop H, et al. A differential evolution algorithm with q-learning for solving engineering design problems. In: 2020 IEEE Congress on Evolutionary Computation (CEC); IEEE; 2020. p. 1–8.
[25] Tian Y, Li X, Ma H, et al. Deep reinforcement learning based adaptive operator selection for evolutionary multi-objective optimization. IEEE Trans Em Top Comput Intell. 2023;7(4):1051–1064. doi: 10.1109/TETCI.2022.3146882
[26] Sun L, Li K. Adaptive operator selection based on dynamic Thompson sampling for MOEA/D. In: Bäck T, Preuss M, Deutz A: editors. Parallel Problem Solving from Nature – PPSN XVI; Cham. Springer International Publishing; 2020. p. 271–284.

[27] Johnn SN, Darvariu VA, Handl J, et al. A graph reinforcement learning framework for neural adaptive large neighbourhood search. Comput Oper Res. 2024;172:106791. doi: 10.1016/j.cor.2024.106791
[28] Karimi-Mamaghan M, Mohammadi M, Pasdeloup B, et al. Learning to select operators in meta-heuristics: an integration of q-learning into the iterated greedy algorithm for the permutation flowshop scheduling problem. Eur J Oper Res. 2023;304(3):1296–1330. doi: 10.1016/j.ejor.2022.03.054
[29] Yi W, Qu R. Automated design of search algorithms based on reinforcement learning. Inf Sci (Ny). 2023;649:119639. doi: 10.1016/j.ins.2023.119639
[30] Durgut R, Aydin ME. Adaptive binary artificial bee colony algorithm. Appl Soft Comput. 2021;101:107054. doi: 10.1016/j.asoc.2020.107054
[31] Koulouriotis DE, Xanthopoulos A. Reinforcement learning and evolutionary algorithms for non-stationary multi-armed bandit problems. Appl Math Comput. 2008;196(2):913–922.
[32] Watkins CJCH, Dayan P. Q-learning. Mach Learn. 1992;8(3-4):279–292.
[33] Tesauro G. Temporal difference learning and td-gammon. Commun ACM. 1995;38(3):58–68. doi: 10.1145/203330.203343
[34] Aydin ME, Öztemel E. Dynamic job-shop scheduling using reinforcement learning agents. Rob Auton Syst. 2000;33(2–3):169–178. doi: 10.1016/S0921-8890(00)00087-7
[35] Fragata I, Blanckaert A, Louro MAD, et al. Evolution in the light of fitness landscape theory. Trends Ecol Evol (Amst). 2019;34(1):69–82. doi: 10.1016/j.tree.2018.10.009
[36] Ochoa G, Malan K. Recent advances in fitness landscape analysis. In: Proceedings of the Genetic and Evolutionary Computation Conference Companion; 2019. p. 1077–1094.
[37] Pitzer E, Affenzeller M. A comprehensive survey on fitness landscape analysis. Rec Adv Intell Eng Syst. 2012;378:161–191. doi: 10.1007/978-3-642-23229-9_8
[38] Teng TH, Handoko SD, Lau HC. Self-organizing neural network for adaptive operator selection in evolutionary search. In: International Conference on Learning and Intelligent Optimization; (Lecture Notes in Computer Science; Vol. 10079); Springer; 2016. p. 187–202.
[39] Karaboga D, Gorkemli B, Ozturk C, et al. A comprehensive survey: artificial bee colony (abc) algorithm and applications. Artif Intell Rev. 2014;42(1):21–57. doi: 10.1007/s10462-012-9328-0
[40] Anescu G, Ulmeanu P. A fast self-adaptive approach to reliability optimization problems. Rev Air Force Acad. 2017;2:23–30. doi: 10.19062/1842-9238
[41] Macias-Escobar T, Cruz-Reyes L, Dorronsoro B, et al. Application of population evolvability in a hyper-heuristic for dynamic multi-objective optimization. Technol Econ Develop Econ. 2019;25(5):951–978. doi: 10.3846/tede.2019.10291
[42] Durgut R, Aydin ME, Ihshaish H, et al. Analysing the predictivity of features to characterise the search space. In: International Conference on Artificial Neural Networks; Springer; 2022. p. 1–13.
[43] Torrey L, Shavlik J. Transfer learning. In: Olivas ES, Guerrero JDM, Martinez-Sober M: editors. Handbook of research on machine learning applications and trends: Algorithms, methods, and techniques. Chapter 11. IGI Global; 2010. p. 242–264.
[44] Pan SJ, Yang Q. A survey on transfer learning. IEEE Trans Knowl Data Eng. 2010;22(10):1345–1359. doi: 10.1109/TKDE.2009.191
[45] Durgut R, Aydin ME, Rakib A. Transfer learning for operator selection: a reinforcement learning approach. Algorithms. 2022;15(1):24. doi: 10.3390/a15010024
[46] Khair U, Lestari YD, Perdana A, et al. Genetic algorithm modification analysis of mutation operators in max one problem. In: 2018 Third International Conference on Informatics and Computing (ICIC); 2018. p. 1–6.
[47] Goldschmidt O, Nehme D, Yu G. Note: on the set-union knapsack problem. Naval Res Logistics (NRL). 1994;41(6):833–842. doi: 10.1002/(ISSN)1520-6750
[48] Ozsoydan FB, Baykasoglu A. A swarm intelligence-based algorithm for the set-union knapsack problem. Future Gen Comput Syst. 2019;93:560–569. doi: 10.1016/j.future.2018.08.002
[49] He Y, Xie H, Wong TL, et al. A novel binary artificial bee colony algorithm for the set-union knapsack problem. Future Gen Comput Syst. 2018;78:77–86. doi: 10.1016/j.future.2017.05.044
[50] Wu C, He Y. Solving the set-union knapsack problem by a novel hybrid jaya algorithm. Soft Comput. 2020;24(3):1883–1902. doi: 10.1007/s00500-019-04021-3

[51] Xiang W, Li Y, He R, et al. Artificial bee colony algorithm with a pure crossover operation for binary optimization. Comput Indus Eng. 2021;152:107011. doi: 10.1016/j.cie.2020.107011
[52] Durgut R. Improved binary artificial bee colony algorithm. Front Inform Technol Electronic Eng. 2021;22(8):1080–1091. doi: 10.1631/FITEE.2000239
[53] Santana Jr CJ, Macedo M, Siqueira H, et al. A novel binary artificial bee colony algorithm. Future Gen Comput Syst. 2019;98:180–196. doi: 10.1016/j.future.2019.03.032

Appendix A

The list of equations used to calculate the features that characterize the problem states, where Equations (A1) to (A11) are related to the population and the rest are related to the individual solutions. The base notation of population-based features is as follows. Let $P = \{p_i | i = 0, 1, \ldots, N\}$ be the set of parent solutions and $C = \{c_i | i = 0, 1, \ldots, N\}$ be the set of children solutions reproduced from P, where each solution has D dimensions. Also, let $F^p = \{f_i^p | i = 0, 1, \ldots, N\}$ be the set of parent fitness values and $F^c = \{f_i^c | i = 0, 1, \ldots, N\}$ be set of children fitness values. g_{best} represents the best solution that has been found so far and p_{best} represents the best solution in the current population.

$$\phi_1 = \frac{\sum_{i=0}^{n-1} \sum_{j=i+1}^{n} \|p_i - p_j\|}{D\frac{n(n-1)}{2}} \tag{A1}$$

$$\phi_2 = \frac{\sum_{i=0}^{n-1} \sum_{j=i+1}^{n} \|f_i^p - f_j^p\|}{\frac{n(n-1)}{2}} \tag{A2}$$

$$\phi_3 = \begin{cases} \frac{|c_i|}{N} & \text{if } f_i^c > f_i^p \\ 0 & \text{otherwise} \end{cases} \tag{A3}$$

$$\phi_4 = \begin{cases} \frac{|c_i|}{N} & \text{if } f_i^c > g_{best} \\ 0 & \text{otherwise} \end{cases} \tag{A4}$$

$$\phi_5 = \begin{cases} \frac{\frac{f_i^c - f_i^p}{f_i^c}}{N} & \text{if } f_i^c > f_i^p \\ 0 & \text{otherwise} \end{cases} \tag{A5}$$

$$\phi_6 = \frac{E[\max(F^c) - \max(F^p)]}{\max(F^p)} \tag{A6}$$

$$\phi_7 = \frac{E[\|x^* - x_t\| - \|x^* - x_{t+1}\|]}{D} \tag{A7}$$

$$\phi_8 = \begin{cases} \sum_{P_{i,j} \in N(P_i)} \frac{\frac{|f^*(P_i) - f(P_{i,j})|}{N}}{\sigma(f(P_i))} & N(P_i) \geq 1 \\ 0 & \text{otherwise} \end{cases} \tag{A8}$$

$$\phi_9 = \phi_4 \times \phi_8 \tag{A9}$$

$$\phi_{10} = \frac{\sum_i^n tn_i}{N} \tag{A10}$$

$$\phi_{11} = \max_{i,j \in \{P,C\}} \|p_i - c_i\| \tag{A11}$$

$$\phi_{12} = \frac{\|x^* - p_i\|}{D} \tag{A12}$$

$$\phi_{13} = \frac{\|p_i - c_i\|}{D} \tag{A13}$$

$$\phi_{14} = \frac{f^{x^*} - f_i^c}{f^{x^*}} \tag{A14}$$

$$\phi_{15} = \frac{f_i^c - f_i^p}{f_i^c} \tag{A15}$$

$$\phi_{16} = \frac{\|p_{best} - p_i\|}{D} \tag{A16}$$

$$\phi_{17} = \frac{\|p_{worst} - p_i\|}{D} \tag{A17}$$

$$\phi_{18} = \frac{\text{trail}_i}{\text{trial}_{max}} \tag{A18}$$

$$\phi_{19} = \frac{osi}{csi} \tag{A19}$$

Appendix B

Algorithm 1 Artificial Bee Colony

```
Initialization of ABC
while Termination criteria is not met do
    Employed Bee Phase
    for each bee in colony do
        Produce and evaluate candidate solution          ▷ operator selected
        if Candidate is better than current bee then
            Replace current bee with candidate
        else
            Increment trial number of current bee
        end if
    end for
    Onlooker Bee Phase
    Calculate selection probability of each bee, p(bee_i)
    for i ← 1 to n do                                    ▷ n is the # of onlooker bees
        Choose current bee according to p(bee_i)
        Produce and evaluate candidate solution          ▷ operator selected
        if Candidate is better than current bee then
            Replace current bee with candidate
        else
            Increment trial number of current bee
        end if
    end for
    Scout Bee Phase
    if trial number of any bee has exceeds the limit then
        Replace the first scout bee with random valid solution
    end if
    Update the best solution
end while
```

Complexity analysis

We analyse the time complexity of the above described Artificial Bee Colony (ABC) algorithm. Let n be the number of bees (or solutions), D be the dimensionality of the problem (the number of decision variables), and T be the number of iterations before the termination criterion is met. The algorithm consists of three phases: Employed Bee Phase, Onlooker Bee Phase, and Scout Bee Phase. We will analyse each phase step by step.

Initialization: During initialization, random solutions are generated for n bees, each with D - dimensional decision variables. The time complexity of this initialization process is: $O(n \cdot D)$. *Employed Bee Phase*: In the employed bee phase, each of the n bees generates and evaluates a candidate solution. For each bee, generating and evaluating the candidate solution takes $O(D)$ time. Since this process is repeated for all n bees, the overall time complexity for this phase is: $O(n \cdot D)$. *Onlooker Bee Phase*: In the onlooker bee phase, the selection probability for each bee is calculated in $O(n)$ time. After that, for each of the n onlooker bees, a candidate solution is produced and evaluated. Similar to the employed bee phase, this takes $O(D)$ time per onlooker bee. Hence, the overall complexity for this phase is: $O(n \cdot D)$. *Scout Bee Phase*: In this phase, the trial numbers of all n bees are checked to determine if any of them have exceeded the predefined limit. This checking step takes $O(n)$ time. If any bee exceeds the limit, a new solution is randomly generated for the scout bee, which requires $O(D)$ time. Thus, the total complexity for this phase is: $O(n + D)$. However, since n generally dominates D, we simplify this to $O(n)$.

Overall Complexity per Iteration: The total time complexity for one iteration of the algorithm is the sum of the complexities of the three phases. The dominant terms are from the employed bee and onlooker bee phases, each of which has a complexity of $O(n \cdot D)$. Therefore, the overall complexity per iteration is: $O(n \cdot D)$. *Total Complexity*: The algorithm runs for T iterations before meeting the termination criterion. Thus, the total time complexity of the algorithm is: $O(T \cdot n \cdot D)$, where T is the number of iterations, n is the number of bees, and D is the dimensionality of the problem.